Technical University of Munich
TUM School of Management
Chair of Logistics and Supply Chain Management

INTERDISCIPLINARY PROJECT · FINAL REPORT

End-to-End Inventory Control via Monotonic Kolmogorov–Arnold Networks

What a certified monotonicity guarantee does, and what it costs

Authors Qi Zhang, Keming Lin
Examiner Prof. Dr. Stefan Minner

Munich, October 2026

Abstract

Neural inventory policies trained end to end through a simulator can learn the ordering decision directly from data, but nothing prevents them from learning decisions a manager would reject, such as ordering more because more stock is already on hand. Such common-sense rules are monotonicity properties, and MonoKAN, a Kolmogorov–Arnold network whose spline edges are projected onto monotone shapes, guarantees them by construction. We study what this guarantee does and what it costs in the lost-sales setting of Maggiar et al. (2025), always against an unconstrained network of the same architecture, trained and evaluated under the same protocol.

In our main design the network decides the order quantity itself from 37 inputs and the guarantee covers all of them. Without it, the same KAN orders more as stock rises on 8.7% to 51.3% of products in each of six training runs; with it, never. The profit comparison depends on how the networks are initialised, so we make it from identical starts. From the start that worked better for both, the certified model earned 14.11 less profit per period than the unconstrained KAN (95% CI 5.90 to 22.32, about 0.3% of profit). From the other start, the unconstrained KAN barely learned to react to stock, and the certified model earned 548 more. The guarantee does not address a second weakness shared by all direct-quantity networks: they lower their order by at most 0.54 units per unit of stock, against 0.90 for the optimal policy. In an alternative design that orders up to a learned quantile of observed demand, the structure itself enforces the stock rule and the guarantee only covers the economic inputs. There we detected no significant difference in mean profit in our final setup (−0.44 per period, 95% CI −1.97 to 1.08), and in a smaller setup matched to the main design the certified model earned 5.90 more per period. The final alternative policy reaches 0.80% regret against the omniscient benchmark, competitive with but not better than classical rules given the same information. These results are consistent with a guarantee whose cost grows with the work it has to do, although the two designs differ in more than the constraint. The alternative design's advantage rests on knowing that the optimal policy is order-up-to, a route Maggiar et al. themselves call fragile.

Contents

1 Introduction 4
2 Setting 4
2.1 The lost-sales inventory problem 4
2.2 Training end to end through the simulator 5
2.3 Measuring performance 5
3 Certified Monotonicity with MonoKAN 5
3.1 Kolmogorov–Arnold networks 5
3.2 Monotone splines and the certificate 5
3.3 Two implementation lessons 6
4 Two Policy Designs 6
4.1 Main design: the network decides the order 6
4.2 Alternative design: order up to a learned target 7
4.3 Baselines and the matched protocol 7
5 Results: Main Design 8
5.1 Without the certificate, the stock rule breaks 8
5.2 Profit with and without the certificate 9
5.3 What the certificate cannot fix 9
6 Results: Alternative Design 10
6.1 From 21.4% to 0.80% regret 10
6.2 Comparison at matched information 10
6.3 The same KAN with and without the certificate 11
6.4 Interpretability, speed and data needs 11
7 The Two Designs Side by Side 12
8 Discussion 13
8.1 Why impose monotonicity if profit is the objective? 13
8.2 The practical questions 13
8.3 Limitations 13
9 Conclusion 14
References 14
Appendix A: Reproducing the Results 15
Appendix B: Protocol Details 15
Appendix C: Additional Results 16

1 Introduction

Every day a retailer decides how many units of each product to order. Too many, and the surplus ties up money and shelf space; too few, and customers leave empty-handed. The classical approach estimates the demand distribution first and then derives the order from it. End-to-end learning skips the separate forecast: a neural network observes the state of the system, including the stock on hand, recent demand and the product's prices and costs, and outputs the order directly, trained to maximise simulated profit. Maggiar et al. (2025) show that such a policy, trained on synthetic products with known optimal benchmarks, comes within 0.41% of the omniscient benchmark in average profit.

A network that learns the whole decision also learns behaviour nobody checked. Nothing in its training stops it from ordering more when more stock is already on hand, or less when the selling price rises. Each such decision may be rare and small, but a manager cannot verify a black box input by input, and a single absurd recommendation is enough to lose trust in a system. These common-sense rules share a mathematical form: when one input rises, the order should move in only one direction. This is *monotonicity*, and it can be guaranteed by the architecture rather than hoped for after training, a long-standing idea in machine learning (Sill, 1998; You et al., 2017). MonoKAN (Polo-Molina et al., 2024) is such an architecture: a Kolmogorov–Arnold network (Liu et al., 2024) whose learned edge functions are constrained to be monotone, which certifies the direction of every designated input for all possible inputs.

The research question of this project is therefore: **what does a certified monotonicity guarantee cost in an end-to-end inventory policy, and where does it actually matter?** We answer it by comparing MonoKAN with an unconstrained KAN of the same architecture, trained and evaluated under the same protocol and started from the same initialisation. In the main design the start turned out to matter, so there we compare the two networks from each of two starts (Section 4.1).

Following the examiner's feedback on our intermediate results, the main design of this report lets the network decide the order quantity itself. In this design the rule "more stock, never a larger order" has to come from the network, so the guarantee does real work. We also report an alternative design, which orders up to a learned target stock. There the structure of the policy already enforces the stock rule, and the guarantee only covers the economic inputs. Comparing both designs shows how the effect of the guarantee differs between them.

The contributions are:

- a direct-quantity MonoKAN policy that certifies the direction of all 37 inputs, and a matched comparison with the same KAN without the certificate and with the feed-forward network of Maggiar et al.;
- a measurement of how often the unconstrained policy breaks the stock rule, and of the profit gap between the certified and the unconstrained model when both start from the same initialisation;
- a soft-quantile target-stock policy with a monotone level network, the steps that took it from 21.4% to 0.80% regret, and comparisons with classical rules that use the same information;
- evidence on the practical promises of MonoKAN: interpretability, model size and speed, and data efficiency.

Section 2 describes the setting and Section 3 the certified network. Section 4 introduces both designs and the protocol. Sections 5 and 6 report the results of each design, Section 7 compares them, Section 8 discusses the practical questions and the limitations, and Section 9 concludes.

2 Setting

2.1 The lost-sales inventory problem

We use the synthetic lost-sales setting of Maggiar et al. (2025). Each product has a selling price p, a purchase cost c, a holding cost h per unit left over at the end of a period and a penalty k per unit of unmet demand. Orders arrive immediately (no lead time) and unmet demand is lost. In period t the policy sees the stock on hand I_t and orders $q_t \geq 0$; then demand d_t is realised, and the period's profit and the next stock are

$$r_t = p\cdot\min(d_t, I_t + q_t) - c\cdot q_t - h\cdot(I_t + q_t - d_t)^+ - k\cdot(d_t - I_t - q_t)^+, \quad I_{t+1} = (I_t + q_t - d_t)^+. \tag{1}$$

Demand is independent and identically distributed over time and follows a Gamma distribution whose mean μ and coefficient of variation ν differ between products. These parameters are hidden: a policy observes only the product's economics and its demand history, which starts with 32 past observations. Products are drawn as in the paper: p exponential with mean 100, $c = p\cdot U$, h exponential with mean 5, $k = 10\cdot U$, μ exponential with mean 100 and ν uniform on (0, 1), with independent uniform draws U.

With no lead time and independent demand, the optimal policy is a myopic base-stock policy (Zipkin, 2000): order up to the critical-fractile quantile of the demand distribution,

$$S^* = F^{-1}(\beta), \quad \beta = \frac{p - c + k}{p - c + k + h}, \quad q_t = (S^* - I_t)^+. \tag{2}$$

This is the newsvendor solution. With the true Gamma parameters it defines the *omniscient* benchmark: a distribution-informed base-stock policy that knows the demand distribution but not the future demand. It is our reference point, not a strict upper bound; on a particular demand path another policy can do as well or better. Classical non-omniscient rules estimate $F^{-1}(\beta)$ from observed demand, either as an empirical quantile or by fitting a Gamma distribution (Section 4.3).

2.2 Training end to end through the simulator

All learned policies are trained by DirectBackprop, as in the paper: the simulator in (1) is written in a differentiable form, a policy runs a batch of products for a 100-period training horizon, and the gradient of the total profit flows through the simulated dynamics into the network's parameters. Each training product has a fixed future demand path, and leftover stock at the end of the horizon is credited at its purchase cost, so that ending the horizon with stock is not penalised. We train with Adam at learning rate 0.001, select the checkpoint with the best validation profit, and repeat every experiment with three seeds (2025, 2026, 2027).

2.3 Measuring performance

Every policy is evaluated on the same 300 validation products and the same demand paths, over 520 periods of which the first 20 are discarded so that the starting stock does not matter. We report two measures.

Profit per period, averaged over products, is the paper's measure. Product economics are heavy tailed (profit per period has mean 4,284 and standard deviation 8,537 over products), so this average is dominated by a few large products.

Regret is the mean per-product shortfall against the omniscient policy, in percent:

$$\text{regret} = \frac{100}{|\mathcal{P}^+|} \sum_{i \in \mathcal{P}^+} \frac{R_i^* - R_i}{R_i^*}, \tag{3}$$

where R_i and R_i^* are the profits of the policy and the omniscient policy on product i. Every product counts the same, so regret reveals failures on typical products that the average hides. The set $\mathcal{P}^+$ excludes the 21 of 300 products on which even the omniscient policy makes no profit, where a relative shortfall is meaningless. The two measures can disagree sharply: our first MonoKAN design trailed a classical rule by only 3.6% in average profit, yet its regret was 21.4%: a typical product lost a fifth of the profit it could have made.

Because all policies face identical products and demand, we compare them *paired by product*, which removes the large product-to-product spread. For a difference in profit we first average each product's profit over the three training seeds and then compare product by product. We report the mean difference with its 95% confidence interval and the paired t-test, the Wilcoxon signed-rank test, which is less sensitive to a few very large products, and the share of products on which one policy is ahead. These tests treat the trained models as given: they describe variation across products, not the variation from retraining with new seeds, which three seeds can only indicate.

3 Certified Monotonicity with MonoKAN

3.1 Kolmogorov–Arnold networks

A multilayer perceptron applies fixed nonlinearities at its nodes and learns linear weights on its edges. A Kolmogorov–Arnold network (KAN) does the reverse: every edge carries its own learnable univariate function, and nodes only sum their incoming edges (Liu et al., 2024). A layer maps $x \in \mathbb{R}^n$ to

$$y_j = \sum_i \varphi_{ji}(x_i), \quad \varphi_{ji}(x) = w^{\mathrm{b}}_{ji} \cdot b(x) + w^{\mathrm{s}}_{ji} \cdot s_{ji}(x), \tag{4}$$

where b is a fixed increasing base function (a shifted, positive ELU in our implementation) and s_{ji} is a learned cubic Hermite spline on a grid of knots, with the value and slope at each knot as parameters. Because each edge is a curve of one variable, a trained KAN can be inspected edge by edge.

3.2 Monotone splines and the certificate

A cubic Hermite spline is monotone if its knot values are monotone and its knot slopes satisfy the condition of Fritsch and Carlson (1980): with Δ the secant slope of an interval and α, β the two knot slopes divided by Δ, the interval is monotone when $\alpha, \beta \geq 0$ and $\alpha^2 + \beta^2 \leq 9$. MonoKAN (Polo-Molina et al., 2024) uses this to certify monotonicity. After every optimiser step we project the parameters back onto the monotone set:

- for every input with a required direction, the first-layer edges get a base weight of that sign, a non-negative spline weight, and a spline projected to be monotone in that direction (knot values made monotone, knot slopes given the right sign and scaled until the Fritsch–Carlson condition holds);
- every edge in later layers is projected to be increasing, with non-negative weights.

A sum of functions that move in one direction moves in that direction, and an increasing function of it keeps it. The network's output is therefore monotone in every designated input, with the designated sign, for every possible input, not only on the training data, up to floating-point rounding. This is a property of the architecture, not a statistical finding.

3.3 Two implementation lessons

Two defects in our first implementation are worth recording, because both changed results. First, the projection wrote the conditions of neighbouring intervals one after the other, although adjacent intervals share a knot slope. The second write could undo the first, leaving rare, tiny violations (worst case $5.6 \cdot 10^{-4}$). The corrected projection combines both conditions at each knot, and a regression test checks it on uneven grids. Second, every spline originally lived on a fixed grid from −2 to 4, while the normalised inputs did not. 63.8% of demand observations fell into the first of seven intervals, and holding cost and lost-sale penalty spanned less than one interval. Those splines could barely bend, and the first model learned an almost constant policy. Placing the knots at quantiles of the training data, the spline analogue of the grid update of Liu et al. (2024), more than halved the regret of our first design (Section 6.1). Knot placement never changes the sign of a slope, so the certificate is unaffected.

4 Two Policy Designs

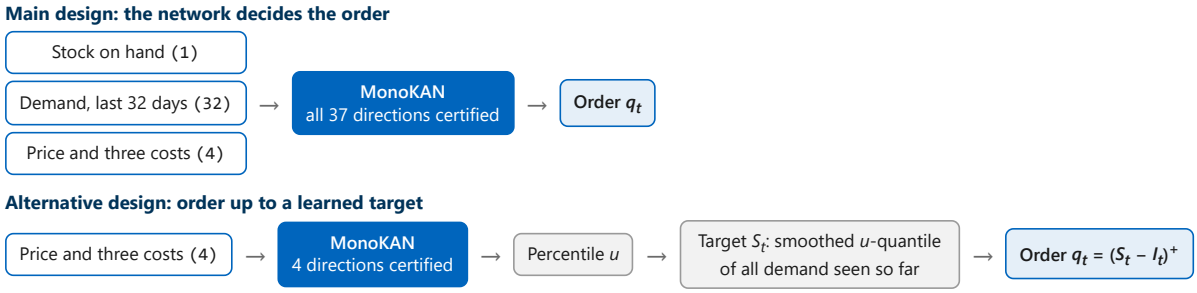


Figure 1. The two policy designs. In the main design the network decides the order, so the rule "more stock, never a larger order" must come from the network. In the alternative it comes from the subtraction in the last step, and the network only chooses a demand percentile from the economics. Blue: learned; grey: fixed computation.

4.1 Main design: the network decides the order

The main design maps everything a manager would look at to the order quantity. The inputs are the stock on hand, the demand of the last 32 periods and the four economic quantities: 1 + 32 + 4 = 37 inputs. We use 32 periods because the paper's setting provides 32 periods of demand history. With orders arriving immediately there is no pipeline stock, so no further state is needed. The rule for each input is fixed in advance (Table 1), and MonoKAN certifies all 37. The demand inputs are sorted before entering the network. Each sorted value is nondecreasing in every raw observation, so the certified directions carry over to the raw inputs. The output passes through a softplus and is multiplied by the mean of the 32 observations available at the start, which makes the order non-negative and proportional to the product's scale without changing any direction. In evaluation, but not in training, each order is also capped so that stock after ordering does not exceed four times that mean. The cap falls with stock and rises with that mean, so it keeps every certified direction, and it applies to all arms alike; Section 5.2 reports its effect.

Table 1. Inputs of the main design and the direction MonoKAN certifies for each.

Input	Count	Certified rule	Reason
Stock on hand	1	more stock → never order more	stock already covers demand
Demand in each of the last 32 periods	32	more demand → never order less	higher demand needs more stock
Selling price	1	higher → never order less	a lost sale forgoes more margin
Lost-sale penalty	1	higher → never order less	running out costs more
Purchase cost	1	higher → never order more	each unit earns less margin
Holding cost	1	higher → never order more	leftover stock costs more

MonoKAN uses one hidden layer of 8 nodes and Hermite splines with 5 intervals placed at data quantiles, 4,265 parameters in total. The comparison KAN has the same architecture and training but no projection. Either network can start from one of two initialisations of its output layer. In the *constant* start the output-layer weights are zero, so every product starts at the same order

per unit of mean demand: the median empirical newsvendor level of the training products. In the *open* start the weights are nonzero, so the initial order already varies with the inputs, and the bias is shifted so that the median initial order on reference contexts equals that level. Our first runs started MonoKAN open and the KAN constant. Because the start turned out to matter, we trained both networks from both starts and compare them from identical starts (Section 5.2). The third arm is the feed-forward network (FCNN) of Maggiar et al. with 2,105 parameters and its own initialisation. All arms are trained on the same 1,000 products for 2,400 epochs to maximise total profit.

To check the stock rule we sweep the stock on hand of every validation product from 0 to 4 days of mean demand in 25 steps, holding all other inputs at the product's observed values, and flag every step at which the order rises (by more than 10^{-4} of a day's demand). The slope of the order against stock over the first 1.5 days measures how strongly a policy reacts. An order-up-to policy has slope −1 until it stops ordering; the omniscient policy averages −0.90 on this sweep.

4.2 Alternative design: order up to a learned target

Equation (2) suggests a different structure: let the network choose only the percentile, and let a fixed formula turn it into an order. A small MonoKAN level network reads only the four economic inputs and outputs a percentile $u = \sigma(f(p, c, h, k))$. The target stock is a smoothed empirical quantile of all demand observed so far. With $z_{(1)} \le \dots \le z_{(n)}$ the sorted observations and $t_j = (j - 1)/(n - 1)$ their positions,

$$S_t = g(u) \cdot \left[z_{(1)} + \sum_{j=2}^{n} \sigma((u - t_j)/\tau) \cdot (z_{(j)} - z_{(j-1)}) \right], \quad q_t = (S_t - I_t)^+, \tag{5}$$

where the sigmoid weights select the order statistics around position u, the width τ is learned, and $g(u) = \exp(a \cdot u + b)$ with $a \ge 0$ is a learned gain that increases with the level. Every term moves the right way. The weights $w_j = \sigma((u - t_j)/\tau)$ lie in (0, 1) and decrease in j, because the positions t_j increase. Rearranged, the bracket in (5) is $(1 - w_2)\, z_{(1)} + \sum_{j=2}^{n-1} (w_j - w_{j+1})\, z_{(j)} + w_n\, z_{(n)}$, a combination of the order statistics with non-negative coefficients; each order statistic is nondecreasing in every observed demand, and the positive gain preserves this, so S_t is nondecreasing in every observed demand. S_t increases with u, which MonoKAN makes increase in price and penalty and decrease in purchase and holding cost. And the subtraction makes the order nonincreasing in stock for *any* level network. The certificate therefore only has to cover four inputs, which is exactly why this design is a useful contrast. The level network has two layers of 8 nodes and 7-interval splines, 1,892 parameters. The final policy is trained on 5,000 products with a scaled objective: each product's simulated profit is divided by $p_i \cdot \hat{\mu}_i \cdot T$, its price times its observed mean demand times the horizon (each floored at 1), before averaging over products, so that a few large products do not dominate the gradient.

The same decoder can use three kinds of demand information, which matters for fair comparisons: one fixed estimate from the first 32 days, the last 32 days updated every period, or all demand seen so far (the final policy). Maggiar et al. (Section 5.3.1) suggest such order-up-to structures, but warn that they are fragile: they only work when the optimal policy is known to be of this form.

4.3 Baselines and the matched protocol

Besides the omniscient policy, we use two classical rules that estimate the target from observed demand: the empirical quantile at the critical fractile, and a Gamma distribution fitted by the method of moments. The Gamma fit is told that demand is Gamma distributed, an advantage no learned policy has. A comparison is only fair if both sides see the same data, so each rule is evaluated with each of the three kinds of information:

- **first 32 days, never updated**: how the paper defines its non-omniscient benchmarks;
- **last 32 days, updated daily**: what a policy in daily use sees, and the demand window of the main design's inputs;
- **all demand seen so far**: the information of the final alternative policy.

Within each comparison, every arm shares the products, demand paths, seeds, horizon, simulator, reward and training budget; for MonoKAN against KAN it also shares the architecture, the parameter count and the initialisation (except in the first main-design runs, which Section 5.2 reports separately). Only the factor under study changes.

5 Results: Main Design

In short. Without the certificate the same KAN breaks the stock rule in every training run, on 9% to 51% of the products. The certificate removes this completely. From the same start, it cost about 14 profit per period (0.3%) when the unconstrained KAN trained well; from the other start the unconstrained KAN barely learned to react to stock, and the certified model earned far more. The certificate does not make the networks react strongly enough to stock, and no direct-quantity network matches a classical rule with the same demand window.

Table 2. Main design, three seeds each, the same 300 validation products. "Start" is the output-layer initialisation (Section 4.1). Profit per period ± standard deviation over seeds. "Breaks" is the share of products on which the order rises with stock at some point of the sweep; "reaction" is the order reduction per unit of stock.

Policy	Start	Profit / period	Regret	Breaks	Reaction	Parameters
MonoKAN (certified)	**constant**	**4,215.20 ± 5.64**	**14.60%**	**0.0%**	**0.46**	**4,265**
Same KAN, no certificate	constant	4,229.31 ± 10.88	13.33%	35.1%	0.54	4,265
MonoKAN (certified)	**open**	**4,206.43 ± 17.56**	**15.71%**	**0.0%**	**0.52**	**4,265**
Same KAN, no certificate	open	3,658.28 ± 147.26	48.78%	23.6%	0.02	4,265
Paper FCNN	—	3,874.38 ± 622.50	20.64%	0.0%	0.34	2,105
Classical rules, last 32 days (empirical quantile / Gamma fit)	—	4,255.93 / 4,275.37	1.90% / 1.25%	0.0%	—	—
Omniscient base stock (knows the true distribution)	—	4,297.45	0.00%	0.0%	0.90	—

5.1 Without the certificate, the stock rule breaks

From the constant start, the KAN without the certificate orders more as stock rises on 35.1% of products on average (Figure 2a). This happens in every training run, but how often varies widely: 45.3%, 8.7% and 51.3% of products for the three seeds. The breaks occur at 3.7% of all swept stock steps; most are small, but the largest is an extra order of about 11% of a day's demand. From the open start the KAN breaks the rule on 19.7%, 16.0% and 35.0% of products, but these breaks are tiny, at most 0.2% of a day's demand, because this KAN barely reacts to stock at all (Section 5.2). Figure 2b shows the largest reversal we found in any run; the break rates above show how common reversals are. Over part of the stock range, the KAN's order goes *up* as stock goes up. The order of the certified MonoKAN on the same product never rises with stock. MonoKAN breaks the rule for no start, seed or product, as its construction guarantees.

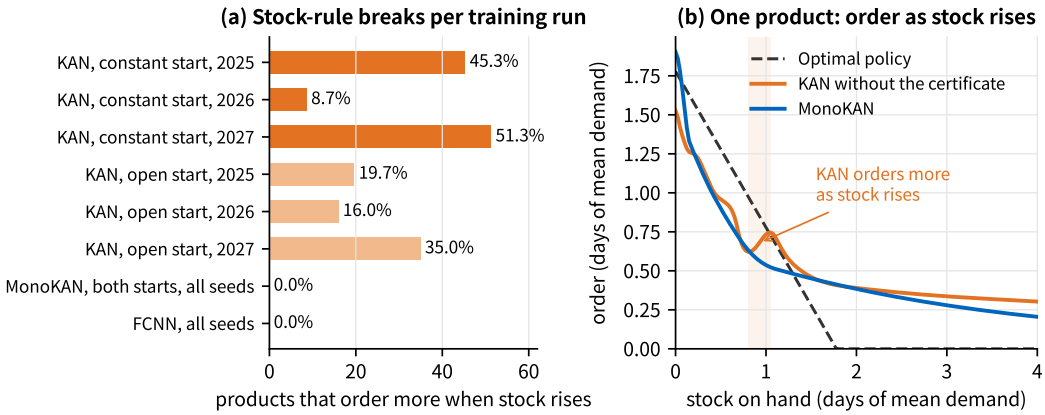


Figure 2. (a) Share of validation products on which the order rises with stock, for each training run of the KAN without the certificate from the constant and the open start, and for MonoKAN (both starts) and the FCNN. (b) Order against stock on hand for the product and run where the KAN breaks the rule most (constant start, seed 2027), with MonoKAN from the open start and the same seed; both axes in days of mean demand. The optimal policy orders up to its target one-for-one.

The risk depends on the model family. The paper's FCNN did not break the rule in any of our three runs, while the KAN did in all six runs, from either start. A practitioner cannot know in advance which kind of model and which training run they will get. The certificate removes that uncertainty: no input combination, seen or unseen, can produce a larger order from more stock. Maggiar et al. (Section 5.3.3) instead add a soft penalty for violations to the training loss, which makes breaks less likely but cannot exclude them.

5.2 Profit with and without the certificate

A fair comparison needs both networks to start from the same initialisation, and the start turned out to matter (Table 3). From the constant start, MonoKAN earns 14.11 per period less than the unconstrained KAN (95% CI 5.90 to 22.32; paired t-test p = 0.0008, Wilcoxon p < 0.0001) and is ahead on only 32% of products. The cost is about 0.3% of profit, and the regret rises from 13.33% to 14.60%. It is the price of all 37 certified directions together; our runs cannot separate the stock rule from the other directions, nor a smaller function class from a harder optimisation.

From the open start the comparison reverses. The unconstrained KAN barely learns to react to stock: its order falls by only 0.02 per unit of stock, against 0.54 from the constant start, its regret is 48.78%, and its profit varies widely between seeds. MonoKAN earns 548.15 more per period (95% CI 390.33 to 705.97) and is ahead on 82% of products. MonoKAN itself depends far less on its start: it earns 8.78 less from the open start than from the constant one (95% CI 0.50 to 17.06), the unconstrained KAN 571 less. We have not investigated why the open start fails for the unconstrained KAN. In these runs the certificate thus did two things: it cost a little profit when both networks started well, and it kept training reliable from a start at which the unconstrained network failed. Our first runs mixed the starts (MonoKAN open, KAN constant) and showed a gap of 22.89; the part beyond the constant-start gap is MonoKAN's own loss from its start.

Table 3. MonoKAN minus the same KAN without the certificate in the main design, profit per period, paired over the 300 products with rewards averaged over seeds.

Start of the two networks	Difference	95% CI	t-test p	Wilcoxon p	MonoKAN ahead
Both constant	−14.11	[−22.32, −5.90]	0.0008	< 0.0001	32%
Both open	+548.15	[+390.33, +705.97]	< 0.0001	< 0.0001	82%
MonoKAN open, KAN constant (first runs)	−22.89	[−32.57, −13.20]	< 0.0001	< 0.0001	28%

The evaluation cap of Section 4.1 does not drive these results. Evaluated without it, MonoKAN trails the KAN by 15.29 from the constant start (95% CI 6.37 to 24.21) and leads by 749 from the open start. Without the cap every network's regret is higher, by 0.8 to 1.6 points for the three that trained well and far more for the open-start KAN, whose excessive orders the cap cuts.

The FCNN's three-seed average of 3,874.38 is misleading. One run (seed 2027) collapsed to 3,155.76 (42.43% regret), while the other two average 4,233.69, slightly above the three-seed average of every KAN arm. The FCNN is less reliable to train rather than simply worse.

5.3 What the certificate cannot fix

Monotonicity fixes the direction of a response, not its strength. An order-up-to policy lowers its order one unit for every unit of stock until it stops ordering, which gives 0.90 on our sweep. Every network reacts far more weakly, at most 0.54 (Figure 3), so the networks keep ordering although enough stock is on hand. This likely explains much of their high regret, and the certificate cannot address it. Maggiar et al. (Section 5.3.2) name this as one reason to set monotonic networks aside: they fix the sign of a reaction but not its size. Our results are consistent with this concern.

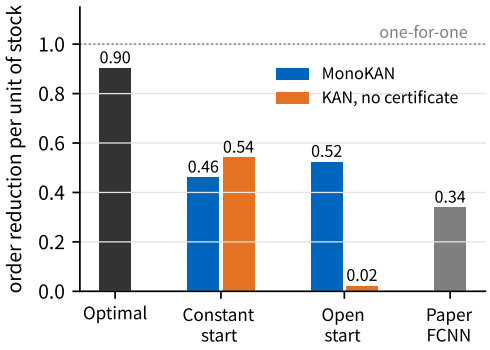


Figure 3. Order reduction per unit of stock over the first 1.5 days of mean demand, averaged over products and seeds, for each network and start.

No direct-quantity network comes close to a classical rule that uses the same 32-day window. The empirical quantile and the Gamma fit on the last 32 days reach 1.90% and 1.25% regret, against 13.33% or more for every network. The paper's interpretability test points the same way. A direct-quantity policy's order at zero stock is its implied order-up-to level. Per unit of mean demand, which removes product scale, MonoKAN's implied level correlates only 0.43 to 0.49 with the optimal level, depending on the start, and the unconstrained KAN's 0.62 from the constant start, against 0.93 for the Gamma fit on the same history. In raw units every policy looks good, because product scale alone explains most of the variation. The certificate makes the direction of every input transparent, but the learned levels are only loosely tied to the optimum.

6 Results: Alternative Design

In short. With the stock rule built into the structure, the final policy reaches 0.80% regret with 1,892 parameters and no rule violations. In every information window it has a lower regret than the same KAN without the certificate. Against classical rules with the same information it is competitive, but not better.

6.1 From 21.4% to 0.80% regret

The first MonoKAN in this design, a deep network that output the target stock directly, reached 21.40% regret, worse than an unconstrained FCNN in the same structure (9.98%) and about twelve times the regret of a classical rule. An unconstrained KAN of the same architecture was similarly poor (20.68%), so the constraint was not the main cause. Figure 4 shows the steps that fixed it, each applied cumulatively. All variants were designed to enforce monotonicity; the projection defect of Section 3.3 was corrected at the fourth step. The two largest gains came from placing the spline knots where the data lies (Section 3.3) and from the soft-quantile structure of equation (5). Using all demand seen so far, weighting every product equally in the loss and training on more products brought the regret to 0.80%. On 3,000 validation products the final policy reaches 0.78%.

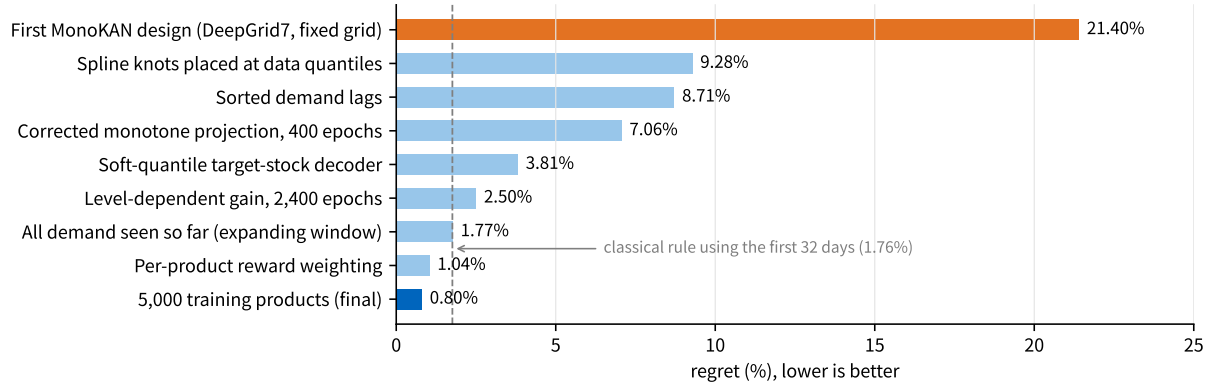


Figure 4. Regret of the alternative design after each cumulative change, three seeds each. All variants were designed to enforce monotonicity; the projection defect of Section 3.3 was corrected at the fourth step.

Several plausible changes did not help (Appendix C). Making the KAN larger made it worse. A warm start that imitated the classical rule before profit training was severely harmful, because the static rule it imitated differs from the expanding-window policy that is deployed. A longer training horizon did not help either.

6.2 Comparison at matched information

A learned policy that uses all demand seen so far must not be compared with a rule that uses only the first 32 days; that would credit the policy with information, not skill. Table 4 and Figure 5 therefore compare every policy within each kind of information. Two patterns stand out. More data helps every policy: for all four, the last column is the best. And within every column the classical Gamma fit is best, although it also knows the demand family. Against the empirical quantile, which is as uninformed as our model, the alternative is worse with the fixed estimate (2.04% against 1.76%), about the same with the last 32 days (1.83% against 1.90%) and worse with all demand (0.80% against 0.27%). It is competitive with good classical rules, not better.

Table 4. Regret of every policy within each kind of information, final setup (5,000 training products), three seeds, the same 300 validation products. The last two rows are the same policy with and without the certificate. * Also told that demand is Gamma distributed.

Policy	First 32 days, never updated	Last 32 days, updated daily	All demand seen so far
Classical Gamma fit *	1.35%	1.25%	0.18%
Classical empirical quantile	1.76%	1.90%	0.27%
Alternative design, KAN without certificate	2.37%	1.96%	0.87%
Alternative design, MonoKAN	**2.04%**	**1.83%**	**0.80%**

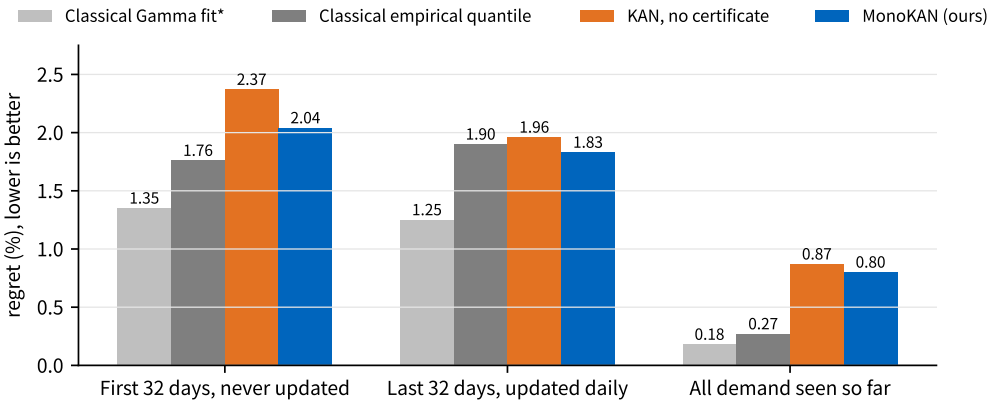


Figure 5. The data of Table 4. The first window is the paper's benchmark definition, the second is what a policy in daily use sees, and the third is the final alternative policy's information.

6.3 The same KAN with and without the certificate

The last two rows of Table 4 are the core test of this design: the same policy, the same 1,892 parameters, the same training and data, once with and once without the certificate. MonoKAN has the lower regret in all three windows. Table 5 adds paired tests on profit. In the final setup with all demand, the mean profit difference is −0.44 per period (95% CI −1.97 to +1.08, p = 0.569): we detected no significant difference, which is not the same as showing that the profits are equal. MonoKAN is ahead on 61% of products (Wilcoxon p = 0.0001). With the last 32 days the gain is small but consistent; with the fixed estimate it is borderline, since only the t-test finds it. In a smaller setup matched to the main design (1,000 products, profit objective, last 32 days) the certified model earned more: +5.90 per period (95% CI +3.32 to +8.48), MonoKAN ahead on 72% of products, and regret 2.26% against 3.35%.

Table 5. MonoKAN minus the same KAN without the certificate in the alternative design, profit per period, paired over the 300 products with rewards averaged over seeds.

Setting	Difference	95% CI	t-test p	Wilcoxon p	MonoKAN ahead	Reading
Final setup, all demand seen so far	−0.44	[−1.97, +1.08]	0.569	0.0001	61%	no significant mean difference; ahead on most products
Final setup, last 32 days	+1.81	[+0.16, +3.46]	0.032	< 0.0001	62%	small but consistent
Final setup, first 32 days	+2.64	[+0.17, +5.10]	0.036	0.464	51%	small and borderline
1,000 products, profit objective, last 32 days	+5.90	[+3.32, +8.48]	< 0.0001	< 0.0001	72%	ahead on both tests

Why does the certificate show no profit cost here, and sometimes help? The subtraction already guarantees the stock rule, so the certificate only asks the level network to follow the four economic directions, which a good policy follows anyway. Our reading is that with ample data the unconstrained KAN finds these directions by itself, while with less data the constraint acts as useful prior knowledge. We have not tested this explanation directly. Two earlier controls point the same way. A parameter-matched MLP level network ties with MonoKAN (−0.08, p = 0.94). In a block with a running-mean decoder, MonoKAN leads the same KAN by +6.73 (p = 0.0001).

6.4 Interpretability, speed and data needs

With only four inputs the level network is easy to read (Figure 6a). The learned target stock rises with price and penalty and falls with purchase and holding cost, as certified. The learned percentile u also tracks the newsvendor fractile β of equation (2) closely, with correlation 0.982 (Figure 6b), although the fractile was never given to the model as an input, an initial value or a training target. This shows agreement in the level, not that the policy equals the textbook one: the quantile-based, order-up-to structure was supplied by design, and the decoder has its own learned smoothing and gain.

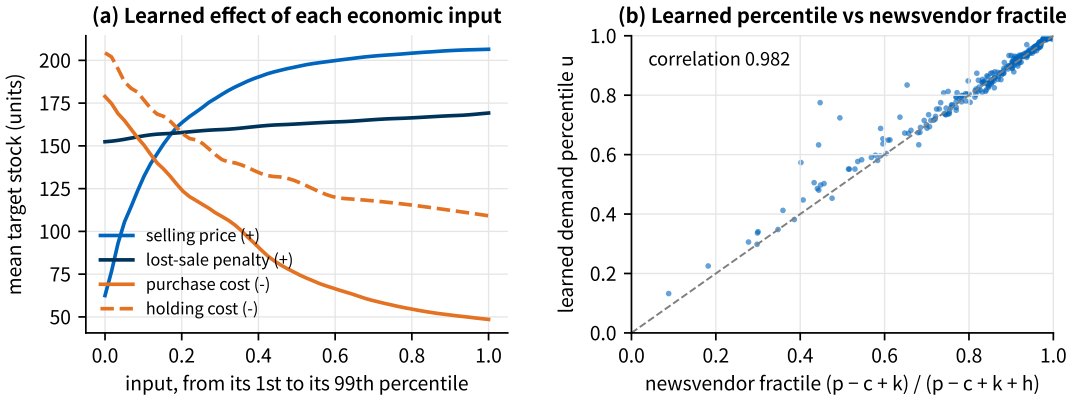


Figure 6. (a) Mean target stock as each economic input sweeps from its 1st to its 99th percentile, with the other inputs at their observed values; the certified direction is in brackets. (b) Learned percentile against the analytic newsvendor fractile for the 300 validation products.

The final policy has 1,892 parameters, against 15,297 for the first design. It is still slower per decision than a feed-forward network of the same size (0.47 ms against 0.06 ms on a CPU), because spline evaluation is more expensive than a matrix product. For daily ordering decisions this does not matter. MonoKAN also does not need less data. With 100 training products it is worse than a parameter-matched MLP (Figure 7). Across the three tested training-set sizes (three seeds each), MonoKAN's regret fell steadily, while the MLP's rose again at 1,000 products; other sizes, budgets or initialisations may behave differently.

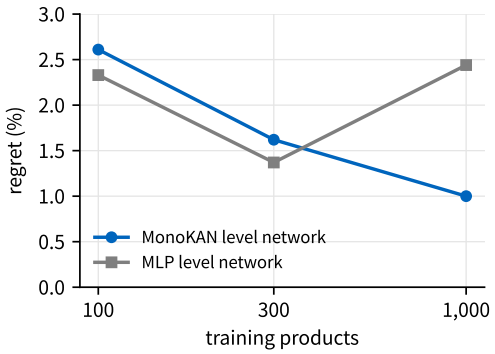


Figure 7. Regret of the alternative design against the number of training products, MonoKAN and a parameter-matched MLP level network.

7 The Two Designs Side by Side

Table 6 puts the two designs next to each other under identical training (1,000 products, 2,400 epochs, profit objective, three seeds) and the same demand window (the last 32 days).

Table 6. The certificate in both designs, matched training and demand window.

	Main design: direct quantity	Alternative: target stock
Who enforces "more stock, never a larger order"	the certificate	the subtraction $(S - I)^+$
Without the certificate: products breaking the stock rule	8.7–51.3% per run	0% (impossible)
Certified minus unconstrained KAN from the same start, profit per period	−14.11 (p = 0.0008) from the constant start; +548.15 from the open start	+5.90 (p < 0.0001)
Regret of MonoKAN	14.60% (constant start), 15.71% (open)	2.26%

The observed effect of the certificate differs between the two designs. In the main design, where it alone enforces the stock rule and the unconstrained network breaks that rule in every run, the certified model earned slightly less from the start that worked better for both networks, and far more from the start at which the unconstrained network failed. In the alternative, where the subtraction enforces the rule, the certified model earned more in this setup, and in the final setup we detected no difference. This is consistent with a guarantee that costs more the more work it has to do, but the comparison does not isolate the stock constraint: the designs also differ in inputs, architecture and decoder. Removing only the stock constraint from the main design would test the idea directly. With the same data and training budget, the alternative's regret is a fraction of the main design's, so the difference does not come from information but from the policy structure and what comes with it. Building inventory knowledge into the policy helped more than anything else we tried; with all demand seen so far, the matched alternative reaches 1.62%.

Why then is the alternative not the main design? Its advantage rests on knowing the shape of the answer in advance. In our setting the optimal policy is order-up-to (equation 2), so a structure that can only produce order-up-to policies loses nothing in principle. Maggiar et al. (Section 5.3.1) call this route fragile, and their own experiments show why. With delivery lead times of 2 to 7 periods, where order-up-to is no longer optimal, order-up-to policies fall 0.79% to 5.68% behind a network that decides the quantity (their Table 4). The direct-quantity design does not impose an order-up-to decoder, although it still makes assumptions: the certified directions, sorted demand inputs, output scaling and limited capacity. It is the more general formulation and the one in which the guarantee has to do the work; we have not verified that it performs better on other inventory problems.

8 Discussion

8.1 Why impose monotonicity if profit is the objective?

Adding a constraint can never improve the optimal profit; at best it leaves it unchanged. In our setting the optimal policy of equation (2) already orders less when stock rises, so at the optimum the stock rule costs nothing. If the optimum could be found exactly and the evaluation products matched the situations met in practice, monotonicity would add nothing to profit. Its role lies elsewhere.

- **Learned policies are approximations.** The optimal policy never orders more when stock rises, yet the unconstrained KAN did so in every training run, on 9% to 51% of products. Training on finite data does not by itself recover the known structure of the optimum.
- **Average profit does not measure every decision.** Profit on the evaluation products says little about states that are rare or absent in training. The certificate holds for every input, which is what lets a planner rely on the direction of each recommendation without checking it case by case.
- **Prior knowledge can help or hurt.** As a restriction it helped in the smaller matched setup of Section 6.3 and kept training reliable from a start at which the unconstrained KAN failed (Section 5.2); where it had to enforce many directions at once, it cost about 0.3% of profit when both networks started well.

We therefore treat monotonicity as a requirement set by the decision maker, comparable to a service-level constraint, and ask what meeting it costs. If profit on the evaluation products were the only criterion, the classical rule would be preferred in the main design, and among the networks the unconstrained KAN, provided it starts well.

8.2 The practical questions

Table 7 summarises our answers to the practical questions about MonoKAN for both designs.

Table 7. Summary of the practical questions.

Question	Main	Evidence	Alternative	Evidence
Does the guarantee cost profit?	Slightly	14 less per period (0.3%) than the KAN from the same constant start; 548 more from the open start, where the KAN fails	None detected	−0.44 (CI −1.97 to +1.08) in the final setup; +5.9 with 1,000 products
Does it always follow the rules?	Yes	0 violations; 9–51% of products per run without the certificate	Yes	0 violations in every check
Good cost performance?	No	14.60–15.71% regret; classical rules with the same data 1.25–1.90%	Partly	0.80% regret; classical rules with the same data 0.18–0.27%
Easy to interpret?	Partly	every direction guaranteed; implied levels match the optimum weakly (0.43–0.49)	Yes	learned percentile tracks the newsvendor fractile (correlation 0.98)
Small and fast?	No	twice the FCNN's parameters; 0.41 vs 0.20 ms per decision	Partly	8× smaller than the first design, slower than an MLP of the same size
Learns from little data?	Not tested	trained at one data size	No	worse than an MLP with 100 products, better with 1,000

8.3 Limitations

- **A simple setting.** Demand is stationary, orders arrive immediately and products are synthetic. This is exactly the setting in which order-up-to is optimal, and it favours the alternative design. We have not tested changing demand, lead times or real data.
- **Scale.** We train on 1,000 to 5,000 products, compared with 40,000 in the paper, and use three seeds per experiment. Our absolute profit levels are therefore not comparable with the paper's.
- **Selection on the validation products.** Checkpoints are selected, and the method was developed, on the same 300 validation products that we report on. Pairing on shared products and demand paths reduces product-level noise, but it does not remove this selection bias. The final alternative policy keeps its standing on 3,000 products (0.78%), but that set contains the original

300 and is not a fully independent test (Appendix C).

- **Two starts, one evaluation cap.** The main design's comparison depends on the initialisation, and we tested two starts; other starts, learning rates or budgets could move the cost of the certificate either way. Its evaluation caps orders, which its training does not (Section 4.1); the cap applies to all arms alike and does not change our conclusions (Section 5.2).
- **The weak stock reaction is unexplained.** All direct-quantity networks react to stock about half as strongly as they should. A plausible cause is that the stock levels visited during training cover only a narrow range; we have not verified this.
- **Model family.** The stock-rule breaks are a property of the KAN family in our runs. The paper's FCNN did not break the rule, so how often an unconstrained model breaks it depends on the model.
- **The main design's training setup** has not yet received the improvements that helped the alternative (5,000 products, per-product weighting), so its absolute performance is likely understated.

9 Conclusion

We asked what a certified monotonicity guarantee costs in an end-to-end inventory policy and where it matters. When the network decides the order itself, the guarantee does real work. Without it, the same network orders more as stock rises on 9% to 51% of the products, in every training run. With MonoKAN this cannot happen. Started from the same initialisation, the certified model earned about 0.3% less when the unconstrained network trained well, and far more from a start at which the unconstrained network barely learned to react to stock. When the structure of the policy already enforces the stock rule, we detected no profit cost of the guarantee, and in a smaller setup the certified model did better. This is consistent with a guarantee whose cost grows with the work it has to do, but our comparison of the designs does not isolate that effect. Building inventory knowledge into the structure produced the best policy of the project, at 0.80% regret; this relies on the shape of the optimal policy being known in advance. The guarantee fixes the direction of every response but not its strength, and in this simple setting a well-chosen classical rule with the same information remains better than both designs.

Three directions follow. First, the main design should receive the improvements that helped the alternative: more training products and per-product weighting. Second, removing only the stock constraint from the main design, while keeping the other 36 directions, would show how much of the remaining cost the stock rule itself causes. Further shape restrictions, such as an order that falls by at most one unit per unit of stock, can also be certified; they bound the reaction between −1 and 0, which the current networks already satisfy, so whether they, or training on a wider range of stock levels, make the reaction stronger remains to be tested. Third, settings with changing demand and lead times, where order-up-to policies are no longer optimal, are where a certified direct-quantity policy should be tested next.

References

Fritsch, F. N., & Carlson, R. E. (1980). Monotone piecewise cubic interpolation. *SIAM Journal on Numerical Analysis*, 17(2), 238–246.

Liu, Z., Wang, Y., Vaidya, S., Ruehle, F., Halverson, J., Soljačić, M., Hou, T. Y., & Tegmark, M. (2024). KAN: Kolmogorov–Arnold networks. arXiv:2404.19756.

Maggiar, A., et al. (2025). Structure-informed deep reinforcement learning for inventory management. arXiv:2507.22040.

Polo-Molina, A., Alfaya, D., & Portela, J. (2024). MonoKAN: Certified monotonic Kolmogorov–Arnold network. arXiv:2409.11078.

Sill, J. (1998). Monotonic networks. In *Advances in Neural Information Processing Systems 10* (pp. 661–667). MIT Press.

You, S., Ding, D., Canini, K., Pfeifer, J., & Gupta, M. (2017). Deep lattice networks and partial monotonic functions. In *Advances in Neural Information Processing Systems 30*.

Zipkin, P. H. (2000). *Foundations of Inventory Management*. McGraw-Hill.

Appendix A: Reproducing the Results

All code is in the accompanying repository. Every command runs from the repository root on a CPU; each target-stock command trains one seed and is repeated for seeds 2025, 2026 and 2027. Saved runs are compared with `scripts/compare_target_stock_runs.py`. The repository's `docs/EXPERIMENTS.md` lists every number with its confidence interval, and `docs/ARCHITECTURE.md` maps the code. The numbers of this report are collected, with their sources, in `report/results.py`. The result files of the twelve KAN runs of the main design are stored in `report/data/direct_quantity` and read when the report is built. The other numbers were transcribed from the experiment outputs; the per-run files of the FCNN and information-window runs were not kept, so the commands below regenerate them rather than read stored files. The final alternative policy's run is kept, and its regret is reproduced by `compare_target_stock_runs.py summary`.

Result	Command

Main design, all five arms (Tables 2, 3, 6; Figures 2, 3)	`python scripts/run_direct_quantity_comparison.py train --out outputs/direct_quantity`
	`python scripts/run_direct_quantity_comparison.py summarize outputs/direct_quantity`
	`python scripts/run_direct_quantity_comparison.py interpret outputs/direct_quantity`
	`python scripts/run_direct_quantity_comparison.py curves outputs/direct_quantity`
	`python scripts/run_direct_quantity_comparison.py cap outputs/direct_quantity`

Final alternative policy	`python scripts/run_soft_quantile_monokan.py --seed 2025 --output-dir outputs/ts_monokan/seed_2025`
Same policy without the certificate	`… --set monokan.level_network=kan`
Last 32 days only / first 32 days only	`… --set monokan.window_mode=sliding` / `… --set monokan.roll_history=false`
Setup matched to the main design (Table 6)	`--config configs/paper_lost_sales_soft_quantile_monokan_profit.yaml --set epochs=2400 --set idp_benchmark.compact.reward_epochs=2400 --set idp_benchmark.compact.validation_interval=150 --set monokan.window_mode=sliding`
Classical rules in every window (Table 4)	`python scripts/run_classical_windows.py`
Regret and paired tests (Tables 4, 5)	`python scripts/compare_target_stock_runs.py summary outputs/ts_monokan outputs/ts_kan` `python scripts/compare_target_stock_runs.py pair outputs/ts_monokan outputs/ts_kan`
Interpretability (Figure 6), certificate audit	`python scripts/run_monokan_interpretability.py …` `python scripts/run_direct_target_structural_audit.py …`
This report	`python report/build_report.py`

Appendix B: Protocol Details

Table B1. Training and evaluation settings.

Setting	Main design	Alternative, final	Alternative, matched
Configuration (`configs/paper_lost_sales_*.yaml`)	`rolling_direct_target_dev_1k_normalized`	`soft_quantile_monokan`	`soft_quantile_monokan_profit`
Training products	1,000	5,000	1,000
Epochs	2,400	2,400	2,400
Objective	total profit	profit, each product weighted equally	total profit
Demand information	last 32 days (order scaled by the initial 32-day mean)	all demand seen so far	last 32 days
Network	KAN 37–8–1, 5 intervals	level network 4–8–8–1, 7 intervals	
Output initialisation	constant or open, both reported (Section 4.1)	the same for MonoKAN and KAN	
Spline knots	placed at quantiles of the training inputs		
Optimiser	Adam, learning rate 0.001, best validation checkpoint		
Training horizon	100 periods, fixed demand paths, terminal stock credited at purchase cost		
Evaluation	300 validation products, 520 periods, first 20 discarded, identical demand paths for all policies		
Order cap	in evaluation only: stock after ordering at most 4 × the initial 32-day mean	none	
Seeds	2025, 2026, 2027		

Appendix C: Additional Results

Table C1. Confirmation on 3,000 validation products, a superset of the 300 used for checkpoint selection. The absolute profit level differs from the main tables because the product draw differs; the ordering of policies is unchanged.

Policy	Profit / period	Regret
Omniscient base stock	4,693.34	0.00%
Soft-quantile MonoKAN, 5,000 products	**4,679.59**	**0.78%**
Soft-quantile MonoKAN, 1,000 products	4,678.87	1.28%
Classical Gamma fit, first 32 days	4,671.74	1.46%
Classical empirical quantile, first 32 days	4,651.87	2.34%

Table C2. Alternative design: profit per period and regret against the number of training products (three seeds, 2,400 epochs each).

Level network	100 products	300 products	1,000 products
MonoKAN	4,279.35 / 2.61%	4,282.60 / 1.62%	4,284.08 / 1.00%
MLP	4,285.05 / 2.33%	4,284.38 / 1.37%	4,278.53 / 2.44%

Table C3. Changes that did not improve the alternative design.

Change	Outcome
Imitation warm start from the classical rule	Severely harmful (8.61% regret after 200 epochs): the imitated rule is static while the deployed policy uses an expanding window.
Larger level network (3 × 16, 12 intervals)	Worse fit of the newsvendor fractile (RMSE 0.033 against 0.026 for 2 × 8).
Training horizon 250 instead of 100	No gain (2.06% regret).
Running mean on top of the expanding window	Worse by 6.8 profit per period: double smoothing.
Per-product normalisation of prices	Rejected: it breaks the certificate, because raising the price moves the cost ratios in conflicting directions.